

```
In [ ]: import marimo as mo
```

Trabalho Prático: Sudoku Genérico como CSP

Contexto

O Sudoku clássico — uma grelha $n^2 \times n^2$ onde cada linha, cada coluna e cada bloco $n \times n$ tem de conter todos os valores de 1 a n^2 sem repetições — é um exemplo canónico de **problema de satisfação de restrições (CSP)**: a "regra" é sempre a mesma (um conjunto de células tem de ter valores todos diferentes), o que muda de linha para linha, de coluna para coluna e de bloco para bloco é apenas **que células pertencem a esse conjunto**.

Isso sugere uma abstração única — um grupo de células com a restrição "todos diferentes", opcionalmente com algumas células já fixas a um valor — a partir da qual linhas, colunas, blocos e ainda outras variantes de Sudoku (diagonais, regiões irregulares, grelhas sobrepostas, etc.) podem ser todas construídas sem duplicar lógica de restrição nenhuma.

Este é um problema de **modelação e resolução de CSP**. Cabe-te a ti escolher a técnica de resolução e justificá-la — o enunciado não fornece código de modelação nem de apresentação de resultados, apenas a interface que o teu notebook tem de expor (secção seguinte) para poder ser testado automaticamente.

Objetivo

Construir, num notebook Marimo, um gerador/resolvedor de Sudoku $n^2 \times n^2$ (com n parametrizável, tipicamente $n = 3$) que:

1. representa qualquer **grupo de células com restrição "todos diferentes"** através de uma classe genérica (secção "`box`" — grupo genérico de células),
2. constrói **linhas, colunas e blocos** como casos particulares dessa classe genérica — os blocos através de uma especialização dedicada a blocos $n \times n$, as linhas e colunas através de uma especialização dedicada a sequências retas de células (secção "`cube`" e `path`"),
3. gera **aleatoriamente** um subconjunto de células já preenchidas (as "pistas" iniciais do puzzle), usando a mesma abstração genérica (secção "Geração aleatória de pistas"),
4. monta o modelo completo (linhas + colunas + blocos + pistas) e o resolve como CSP, devolvendo a grelha preenchida ou sinalizando que não há solução (secção "Resolução").

Requisitos obrigatórios

O teu notebook tem de expor, com este comportamento, os seguintes elementos (os nomes propostos abaixo são sugestões que facilitam a correção automática — podes usar outros, desde que documentes a correspondência):

`box` — grupo genérico de células (R1)

Uma classe que representa **qualquer** conjunto de células da grelha às quais se aplica a restrição "todos os valores diferentes", com algumas delas possivelmente já fixas:

- guarda internamente uma associação `(linha, coluna) → valor` ou `None` (`None` = célula livre; um inteiro = célula fixa/pinada a esse valor);
- um construtor que aceita opcionalmente esse conjunto inicial de células (vazio por omissão);
- um método `add(i, j, val=None)` que acrescenta a célula `(i, j)` ao grupo, opcionalmente fixando-a a `val`, e que **rejeita** (levanta exceção) coordenadas fora da grelha ou valores fora do intervalo $[1, n^2]$;
- uma forma de obter a representação do grupo como matriz $n^2 \times n^2$, com zeros nas células não pertencentes ao grupo ou não fixas, e o valor fixo nas restantes.

Esta classe **não deve saber nada** sobre linhas, colunas, blocos ou Sudoku — só sabe lidar com "um conjunto de células, algumas fixas". Essa generalidade é o que te vai permitir, mais tarde, tratar da mesma forma linhas, colunas, blocos, pistas aleatórias e (nas extensões opcionais) diagonais ou regiões irregulares.

`cube` e `path` — duas formas concretas de grupo (R2, R3)

A partir da classe genérica, define duas especializações:

- **R2.** Um grupo que representa o **bloco** $n \times n$ cujo canto superior esquerdo é a célula $(i \cdot n, j \cdot n)$, parametrizado pelos índices de bloco (i, j) com $0 \leq i, j < n$.
- **R3.** Um grupo que representa o **troço reto** (horizontal ou vertical) de células entre duas coordenadas `inicio` e `fim`, inclusive — tem de funcionar tanto para `fim` "depois" de `inicio` como "antes" (ou seja, percorrer a sequência em qualquer sentido).

Geração aleatória de pistas (R4)

Uma função que devolve um grupo (`box`) com k células escolhidas aleatoriamente na grelha, cada uma fixa a um valor também escolhido aleatoriamente em $[1, n^2]$ (k deve ter um valor por omissão razoável, por exemplo da ordem de n). Repara que esta função **não precisa de nenhuma classe nova** — o resultado é, de novo, apenas um `box`.

Modelo e resolução (R5, R6)

- **R5.** Um modelo de CSP para a grelha $n^2 \times n^2$, com uma variável inteira por célula, cada uma no intervalo $[1, n^2]$; um método que recebe **um número arbitrário de grupos** (`box` , `cube` , `path` , ou pistas aleatórias — o modelo não deve distinguir a sua origem) e, para cada um, impõe que as suas células sejam todas diferentes e fixa as que tiverem valor atribuído; e um método de resolução que devolve a grelha preenchida ou sinaliza, de forma distinguível, que o puzzle não tem solução.
- **R6.** Um Sudoku $n^2 \times n^2$ completo é montado juntando: todas as linhas, todas as colunas, todos os blocos $n \times n$ e (pelo menos) um grupo de pistas aleatórias — e resolvido.

Como testar/validar

O teu notebook (ou um ficheiro de testes à parte) tem de verificar automaticamente, para uma grelha resolvida:

- que cada linha, cada coluna e cada bloco $n \times n$ contém exatamente os valores $1 \dots n^2$, sem repetições;
- que as células fixadas pelas pistas aleatórias mantêm, na solução, o valor com que foram fixadas;
- que `add` (ou equivalente) rejeita coordenadas fora da grelha e valores fora de $[1, n^2]$.

Corre o fluxo completo (gerar pistas aleatórias → montar linhas + colunas + blocos + pistas → resolver → validar) pelo menos uma vez com $n = 3$ (Sudoku clássico 9×9) e confirma que também funciona com outro valor de n (ex.: $n = 2$, grelha 4×4), para garantir que nada está fixo a 9×9 no teu código.

O que é deixado ao teu critério

O enunciado define **que abstrações** o notebook tem de expor e **que comportamento** têm de ter, não **como** as deves implementar. Ficam ao teu critério, desde que justificadas no notebook:

- a técnica e biblioteca de resolução do CSP (CP-SAT do OR-Tools é a sugestão da disciplina, mas és livre de escolher outra abordagem de Lógica Computacional, justificando a escolha);
- a estrutura de dados interna do grupo genérico (dicionário, matriz esparsa, etc.);
- a forma de apresentar a grelha resultante (texto, tabela, `mo.ui` , gráfico — o que achares mais claro);
- o comportamento exato quando o puzzle gerado aleatoriamente não tem solução (podes, por exemplo, tentar novas pistas aleatórias até obteres um puzzle solúvel, ou simplesmente reportar o insucesso — justifica a escolha).

Extensões opcionais (bónus)

A generalidade do `box` é o que torna estas extensões possíveis sem tocar no modelo CSP em si — cada uma acrescenta apenas **novos grupos** de células:

- **Sudoku diagonal (X-Sudoku)**: acrescenta um grupo (`box` , sem precisar de nova subclasse) para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".
- **Sudoku irregular (jigsaw)**: substitui os blocos $n \times n$ regulares por regiões de forma arbitrária mas do mesmo tamanho, cada uma representada como um `box` construído célula a célula em vez de por `cube` .
- **Hyper-Sudoku / Windoku**: acrescenta 4 blocos extra (também `box` , de forma semelhante a `cube` mas sem estarem alinhados com a grelha $n \times n$ de blocos) sobrepostos aos existentes.
- **Escala**: mostra que o teu código funciona (talvez mais devagar) para $n = 6$ (grelha 36×36) sem alterações, e discute os limites de desempenho que encontre.
- **Sudoku tridimensional** define a estrutura de "boxes" numa grelha $n^2 \times n^2 \times n^2$.

Solução desenvolvida

A partir deste ponto apresenta-se a formulação, implementação e validação adotadas pelo grupo para satisfazer os requisitos R1–R6 do enunciado.

Formulação adotada

Seja $n \geq 1$ e seja

$$N = n^2$$

a dimensão da grelha. As posições possíveis são

$$G = \{0, \dots, N-1\} \times \{0, \dots, N-1\}.$$

R1 — grupo genérico de células

Um `box` representa um subconjunto $B \subseteq G$. Cada célula pertencente ao grupo está associada a um valor

$$v(i, j) \in \{1, \dots, N\} \cup \{\text{None}\}.$$

`None` significa que a célula pertence ao grupo mas não está fixada; um inteiro significa que essa célula está fixada a esse valor.

R2 — blocos

Para índices a, b , com $0 \leq a, b < n$, o bloco `cube(n, a, b)` contém

$$C_{a,b} = \{(a \cdot n + r, b \cdot n + c) \mid 0 \leq r, c < n\}.$$

Assim, cada bloco contém exatamente n^2 células.

R3 — linhas e colunas

Um `path` representa todas as células de um percurso horizontal ou vertical entre duas coordenadas, incluindo as duas extremidades. O percurso pode ser feito em qualquer dos sentidos.

R4 — pistas aleatórias

É escolhido um conjunto de k células distintas

$$P \subseteq G$$

e, para cada $(i, j) \in P$, é escolhido um valor aleatório

$$v_{i,j} \in \{1, \dots, N\}.$$

As pistas são representadas através da mesma abstração `box`.

R5 — modelo CSP

Para cada célula da grelha é criada uma variável inteira

$$x_{i,j} \in \{1, \dots, N\}.$$

Para cada grupo B fornecido ao modelo é imposta a restrição

$$\text{AllDifferent}(\{x_{i,j} \mid (i, j) \in B\}).$$

Se uma célula do grupo tiver um valor fixo v , é ainda acrescentada a igualdade

$$x_{i,j} = v.$$

O modelo recebe todos os grupos da mesma forma, independentemente de terem sido construídos por `box`, `cube`, `path` ou pela geração aleatória de pistas.

R6 — Sudoku completo

Um Sudoku completo é construído reunindo:

- todas as N linhas;
- todas as N colunas;
- todos os n^2 blocos;
- pelo menos um grupo de pistas.

O problema é de satisfação: não existe função objetivo. Pretende-se encontrar uma atribuição que satisfaça todas as restrições ou sinalizar que o conjunto de restrições é inviável.

Para a implementação foi utilizado OR-Tools CP-SAT, seguindo a sugestão do enunciado para a resolução do CSP.

Decisões de implementação

Para concretizar esta formulação foram adotadas as seguintes decisões:

- `box` utiliza internamente um dicionário `(linha, coluna) → valor ou None`, por corresponder diretamente à associação definida no enunciado;
- `n` é passado explicitamente a cada grupo, evitando depender de uma dimensão global;
- tentar adicionar novamente uma coordenada ao mesmo `box` é tratado como erro através de `ValueError`;
- um `path` com início igual ao fim é aceite como um percurso de uma única célula, enquanto percursos não horizontais nem verticais são rejeitados;
- na geração aleatória de pistas é usado `k = n` por omissão e é possível fornecer uma `seed` para tornar as experiências reproduzíveis;
- se as pistas aleatórias originarem um CSP inviável, o programa reporta `INFEASIBLE` em vez de gerar silenciosamente novas pistas;
- foi utilizado OR-Tools CP-SAT por suportar diretamente variáveis inteiras com domínio finito e a restrição `AllDifferent`, adequada à formulação CSP deste problema.

Estas opções são decisões de implementação do grupo e não requisitos adicionais do enunciado.

```
In [ ]: # =====
# Contribuição LLM:
# Implementação da classe genérica `box`, desenvolvida a partir
# da formalização e das decisões discutidas no diálogo LLM
# associado ao relatório do TP1.2.
# =====

class box:
    """
    Grupo genérico de células de uma grelha n^2 x n^2.

    Cada coordenada pertencente ao grupo está associada a:
    - None, se a célula estiver livre;
    - um inteiro entre 1 e n^2, se estiver fixada.
    """

    def __init__(self, n, cells=None):
        if not isinstance(n, int) or n <= 0:
            raise ValueError("n deve ser um inteiro positivo.")
```

```

self.n = n
self.size = n ** 2
self._cells = {}

if cells is not None:
    for (i, j), val in cells.items():
        self.add(i, j, val)

def add(self, i, j, val=None):
    """
    Acrescenta uma célula ao grupo, opcionalmente com valor fixo.
    """

    if not isinstance(i, int) or not isinstance(j, int):
        raise ValueError("As coordenadas devem ser inteiros.")

    if not (0 <= i < self.size and 0 <= j < self.size):
        raise ValueError("Coordenada fora da grelha.")

    if (i, j) in self._cells:
        raise ValueError("A célula já pertence ao grupo.")

    if val is not None:
        if not isinstance(val, int) or not (1 <= val <= self.size):
            raise ValueError(
                f"O valor deve pertencer ao intervalo [1, {self.size}]."
            )

    self._cells[(i, j)] = val

@property
def cells(self):
    """
    Devolve uma cópia da associação interna de células.
    """
    return dict(self._cells)

def to_matrix(self):
    """
    Representa o grupo como uma matriz n^2 x n^2.

    Células não pertencentes ao grupo e células livres
    são representadas por 0.
    """

    matrix = [
        [0 for _ in range(self.size)]
        for _ in range(self.size)
    ]

    for (i, j), val in self._cells.items():
        if val is not None:
            matrix[i][j] = val

    return matrix

```

```

In [ ]: # =====
# Contribuição LLM:
# Testes automáticos básicos do requisito R1 (`box`).
# =====

# 1. Criação vazia
b = box(2)
assert b.cells == {}

# 2. Construtor com células iniciais
b_inicial = box(2, {
    (0, 0): None,
    (1, 2): 3
})

assert b_inicial.cells[(0, 0)] is None
assert b_inicial.cells[(1, 2)] == 3

# 3. Célula livre
b.add(0, 1)
assert b.cells[(0, 1)] is None

# 4. Célula fixa
b.add(2, 3, 4)
assert b.cells[(2, 3)] == 4

# 5. Representação matricial
m = b_inicial.to_matrix()

assert len(m) == 4
assert all(len(row) == 4 for row in m)

assert m[0][0] == 0      # célula livre
assert m[1][2] == 3      # célula fixa
assert m[3][3] == 0      # célula fora do grupo

# Função auxiliar apenas para os testes de exceção
def deve_lancar_value_error(func):
    try:
        func()
        assert False, "Era esperado ValueError."
    except ValueError:
        pass

# 6. Coordenadas inválidas
deve_lancar_value_error(lambda: b.add(-1, 0))
deve_lancar_value_error(lambda: b.add(4, 0))
deve_lancar_value_error(lambda: b.add(0, 4))

# 7. Valor inválido
deve_lancar_value_error(lambda: b.add(1, 1, 0))
deve_lancar_value_error(lambda: b.add(1, 1, 5))

```

```
# 8. Coordenada duplicada
deve_lancar_value_error(lambda: b.add(0, 1))

print("R1: testes concluídos com sucesso")
```

R1: testes concluídos com sucesso

```
In [ ]: # =====
# Contribuição LLM:
# Implementação das especializações `cube` e `path`
# correspondentes aos requisitos R2 e R3, desenvolvida a partir
# da formalização discutida no diálogo LLM do TP1.2.
# =====

class cube(box):
    """
    Bloco n x n de uma grelha n^2 x n^2.

    (i, j) são os índices do bloco, com 0 <= i, j < n.
    """

    def __init__(self, n, i, j):
        super().__init__(n)

        if not isinstance(i, int) or not isinstance(j, int):
            raise ValueError("Os índices do bloco devem ser inteiros.")

        if not (0 <= i < n and 0 <= j < n):
            raise ValueError("Índice de bloco fora da grelha.")

        for r in range(n):
            for c in range(n):
                self.add(i * n + r, j * n + c)

class path(box):
    """
    Troço horizontal ou vertical, inclusive, entre duas coordenadas.
    """

    def __init__(self, n, inicio, fim):
        super().__init__(n)

        r1, c1 = inicio
        r2, c2 = fim

        if r1 == r2:
            # Percurso horizontal
            passo = 1 if c2 >= c1 else -1

            for c in range(c1, c2 + passo, passo):
                self.add(r1, c)

        elif c1 == c2:
            # Percurso vertical
            passo = 1 if r2 >= r1 else -1
```

```

        for r in range(r1, r2 + passo, passo):
            self.add(r, c1)

    else:
        raise ValueError(
            "Um path tem de ser horizontal ou vertical."
        )

```

```

In [ ]: # =====
# Contribuição LLM:
# Testes automáticos dos requisitos R2 (`cube`) e R3 (`path`).
# Reutiliza `deve_lancar_value_error`, definida nos testes de R1.
# =====

def testar_r2_r3():
    # -----
    # R2 - cube
    # -----

    # Bloco superior direito numa grelha 4 x 4
    c1 = cube(2, 0, 1)

    assert isinstance(c1, box)

    assert set(c1.cells.keys()) == {
        (0, 2), (0, 3),
        (1, 2), (1, 3)
    }

    assert all(val is None for val in c1.cells.values())
    assert len(c1.cells) == 4

    # Outro bloco: inferior direito
    c2 = cube(2, 1, 1)

    assert set(c2.cells.keys()) == {
        (2, 2), (2, 3),
        (3, 2), (3, 3)
    }

    # Índices de bloco inválidos
    deve_lancar_value_error(lambda: cube(2, -1, 0))
    deve_lancar_value_error(lambda: cube(2, 2, 0))
    deve_lancar_value_error(lambda: cube(2, 0, 2))

    # -----
    # R3 - path
    # -----

    # Horizontal crescente
    p1 = path(2, (1, 0), (1, 3))

```

```

assert isinstance(p1, box)

assert set(p1.cells.keys()) == {
    (1, 0),
    (1, 1),
    (1, 2),
    (1, 3)
}

# Horizontal decrescente
p2 = path(2, (1, 3), (1, 0))

assert set(p2.cells.keys()) == {
    (1, 0),
    (1, 1),
    (1, 2),
    (1, 3)
}

# Vertical crescente
p3 = path(2, (0, 2), (3, 2))

assert set(p3.cells.keys()) == {
    (0, 2),
    (1, 2),
    (2, 2),
    (3, 2)
}

# Vertical decrescente
p4 = path(2, (3, 2), (0, 2))

assert set(p4.cells.keys()) == {
    (0, 2),
    (1, 2),
    (2, 2),
    (3, 2)
}

# inicio == fim
p5 = path(2, (2, 2), (2, 2))

assert set(p5.cells.keys()) == {
    (2, 2)
}

# Percurso que não é horizontal nem vertical
deve_lancar_value_error(
    lambda: path(2, (0, 0), (2, 2))
)

```

```

# Coordenadas fora da grelha
deve_lancar_value_error(
    lambda: path(2, (0, 0), (0, 4))
)

deve_lancar_value_error(
    lambda: path(2, (-1, 1), (2, 1))
)

print("R2 e R3: testes concluídos com sucesso")

testar_r2_r3()

```

R2 e R3: testes concluídos com sucesso

```

In [ ]: # =====
# Contribuição LLM:
# Implementação da geração aleatória de pistas correspondente
# ao requisito R4, segundo a formalização e decisões discutidas
# no diálogo LLM do TP1.2.
# =====

import random

def gerar_pistas(n, k=None, seed=None):
    """
    Gera um box com k células distintas escolhidas aleatoriamente.

    Cada célula recebe um valor aleatório entre 1 e n^2.
    Se k não for indicado, é usado k = n.
    """

    pistas = box(n)

    if k is None:
        k = n

    total_celulas = pistas.size ** 2

    if not isinstance(k, int) or k < 0 or k > total_celulas:
        raise ValueError(
            f"k deve ser um inteiro entre 0 e {total_celulas}."
        )

    rng = random.Random(seed)

    coordenadas = [
        (i, j)
        for i in range(pistas.size)
        for j in range(pistas.size)
    ]

    escolhidas = rng.sample(coordenadas, k)

```

```

for i, j in escolhidas:
    val = rng.randint(1, pistas.size)
    pistas.add(i, j, val)

return pistas

```

```

In [ ]: # =====
# Contribuição LLM:
# Testes automáticos do requisito R4 - geração aleatória
# e reproduzível de pistas.
# =====

def testar_r4():
    # -----
    # Resultado e número de pistas
    # -----

    p1 = gerar_pistas(2, k=5, seed=123)

    assert isinstance(p1, box)
    assert len(p1.cells) == 5

    # -----
    # Coordenadas e valores válidos
    # -----

    for (i, j), val in p1.cells.items():
        assert 0 <= i < 4
        assert 0 <= j < 4
        assert 1 <= val <= 4

    # As coordenadas são distintas
    assert len(set(p1.cells.keys())) == 5

    # -----
    # Reprodutibilidade com a mesma semente
    # -----

    p2 = gerar_pistas(2, k=5, seed=123)

    assert p1.cells == p2.cells

    # -----
    # Valor por omissão: k = n
    # -----

    p_default = gerar_pistas(3, seed=123)

    assert len(p_default.cells) == 3

```

```

# -----
# Valores inválidos de k
# -----

deve_lancar_value_error(
    lambda: gerar_pistas(2, k=-1, seed=123)
)

deve_lancar_value_error(
    lambda: gerar_pistas(2, k=17, seed=123)
)

print("R4: testes concluídos com sucesso")

testar_r4()

```

R4: testes concluídos com sucesso

```

In [ ]: # =====
# Contribuição LLM:
# Implementação do modelo CSP correspondente ao requisito R5,
# usando OR-Tools CP-SAT, a partir da formalização discutida
# no diálogo LLM do TP1.2.
# =====

from ortools.sat.python import cp_model

class sudoku_csp:
    """
    Modelo CSP para uma grelha n^2 x n^2.
    """

    def __init__(self, n):
        self.n = n
        self.size = n ** 2

        self.model = cp_model.CpModel()

        # Uma variável inteira por célula, com domínio [1, n^2].
        self.x = [
            [
                self.model.NewIntVar(
                    1,
                    self.size,
                    f"x_{i}_{j}"
                )
                for j in range(self.size)
            ]
            for i in range(self.size)
        ]

    def add_groups(self, *groups):
        """
        Acrescenta ao modelo um número arbitrário de grupos.

```

```

Para cada grupo:
    - impõe AllDifferent às células pertencentes ao grupo;
    - transforma cada valor fixo numa igualdade.
"""

for group in groups:
    if group.n != self.n:
        raise ValueError(
            "O grupo e o modelo devem usar o mesmo valor de n."
        )

    variables = [
        self.x[i][j]
        for (i, j) in group.cells.keys()
    ]

    if len(variables) > 1:
        self.model.AddAllDifferent(variables)

    for (i, j), val in group.cells.items():
        if val is not None:
            self.model.Add(
                self.x[i][j] == val
            )

def solve(self):
    """
    Resolve o CSP.

    Devolve:
    - uma matriz  $n^2 \times n^2$  se existir uma solução;
    - None se o modelo for INFEASIBLE.

    Outros estados do solver não são confundidos com inviabilidade.
    """

    solver = cp_model.CpSolver()
    status = solver.Solve(self.model)

    if status in (
        cp_model.FEASIBLE,
        cp_model.OPTIMAL
    ):
        return [
            [
                solver.Value(self.x[i][j])
                for j in range(self.size)
            ]
            for i in range(self.size)
        ]

    if status == cp_model.INFEASIBLE:
        return None

    raise RuntimeError(

```

```
)  
"O solver terminou sem determinar satisfatibilidade."
```

```
In [ ]: # =====  
# Contribuição LLM:  
# Testes automáticos do requisito R5:  
# solução matricial, AllDifferent, valores fixos,  
# grupos arbitrários, incompatibilidade de n e inviabilidade.  
# =====  
  
# -----  
# 1. Caso satisfazível  
# -----  
  
csp = sudoku_csp(2)  
  
# Três grupos de origens diferentes:  
# path, cube e box de pistas.  
linha = path(2, (0, 0), (0, 3))  
bloco = cube(2, 0, 0)  
  
pistas = box(2, {  
    (0, 0): 1,  
    (1, 0): 2  
})  
  
# O modelo recebe todos da mesma forma.  
csp.add_groups(linha, bloco, pistas)  
  
solucao = csp.solve()  
  
# Deve existir uma solução.  
assert solucao is not None  
  
# A solução devolvida deve ser uma matriz 4 x 4.  
assert isinstance(solucao, list)  
assert len(solucao) == 4  
assert all(isinstance(row, list) for row in solucao)  
assert all(len(row) == 4 for row in solucao)  
  
# -----  
# 2. AllDifferent  
# -----  
  
# A primeira linha pertence ao grupo `linha`, que contém  
# exatamente quatro variáveis com domínio {1, 2, 3, 4}.  
assert set(solucao[0]) == {1, 2, 3, 4}  
  
# O bloco superior esquerdo também recebeu AllDifferent.  
valores_bloco = [  
    solucao[0][0],
```

```

        solucao[0][1],
        solucao[1][0],
        solucao[1][1]
    ]

    assert len(set(valores_bloco)) == 4

# -----
# 3. Valores fixos preservados
# -----

    assert solucao[0][0] == 1
    assert solucao[1][0] == 2

# -----
# 4. Grupo com n incompatível
# -----

    grupo_incompativel = box(3)

    deve_lancar_value_error(
        lambda: csp.add_groups(grupo_incompativel)
    )

# -----
# 5. CSP deliberadamente contraditório
# -----

    csp_inviavel = sudoku_csp(2)

    linha_inviavel = path(
        2,
        (0, 0),
        (0, 3)
    )

    # AllDifferent exige que estas duas células tenham valores
    # distintos, mas as pistas fixam ambas ao valor 1.
    pistas_incompativeis = box(2, {
        (0, 0): 1,
        (0, 1): 1
    })

    csp_inviavel.add_groups(
        linha_inviavel,
        pistas_incompativeis
    )

    resultado_inviavel = csp_inviavel.solve()

    # solve() só devolve None quando o estado é INFEASIBLE.
    # Qualquer outro estado originaria RuntimeError.
    assert resultado_inviavel is None

```

```
print("R5: testes concluídos com sucesso")
```

R5: testes concluídos com sucesso

```
In [ ]: # =====
# Contribuição LLM:
# Implementação de R6 - construção de um Sudoku completo
#  $n^2 \times n^2$  através de path, cube, pistas e do modelo genérico
# sudoku_csp desenvolvido em R5.
# =====

def resolver_sudoku(n, k=None, seed=None):
    """
    Constrói e resolve um Sudoku  $n^2 \times n^2$ .

    O modelo contém:
    - todas as linhas;
    - todas as colunas;
    - todos os blocos  $n \times n$ ;
    - um grupo de pistas aleatórias.

    Devolve:
    - a solução, ou None se o CSP for INFEASIBLE;
    - o box de pistas usado na construção.
    """

    size = n ** 2

    # Todas as Linhas
    linhas = [
        path(n, (i, 0), (i, size - 1))
        for i in range(size)
    ]

    # Todas as colunas
    colunas = [
        path(n, (0, j), (size - 1, j))
        for j in range(size)
    ]

    # Todos os blocos  $n \times n$ 
    blocos = [
        cube(n, i, j)
        for i in range(n)
        for j in range(n)
    ]

    # Grupo de pistas aleatórias
    pistas = gerar_pistas(
        n,
        k=k,
        seed=seed
    )
```

```

# Modelo CSP genérico de R5
modelo = sudoku_csp(n)

modelo.add_groups(
    *(linhas + colunas + blocos + [pistas])
)

solucao = modelo.solve()

return solucao, pistas

```

```

In [ ]: # =====
# Contribuição LLM:
# Validador independente da solução de R6.
# Verifica diretamente dimensão, linhas, colunas, blocos
# e preservação das pistas, sem consultar o solver.
# =====

def validar_sudoku(solucao, n, pistas):
    """
    Verifica independentemente uma solução de Sudoku.

    Devolve True apenas se:
    - a matriz tiver dimensão  $n^2 \times n^2$ ;
    - todas as linhas contiverem exatamente  $1..n^2$ ;
    - todas as colunas contiverem exatamente  $1..n^2$ ;
    - todos os blocos  $n \times n$  contiverem exatamente  $1..n^2$ ;
    - todas as pistas forem preservadas.
    """

    if solucao is None:
        return False

    size = n ** 2
    valores_esperados = set(range(1, size + 1))

    # -----
    # Dimensão  $n^2 \times n^2$ 
    # -----

    if len(solucao) != size:
        return False

    if any(len(linha) != size for linha in solucao):
        return False

    # -----
    # Linhas
    # -----

    for i in range(size):
        if set(solucao[i]) != valores_esperados:
            return False

    # -----
    # Colunas

```

```

# -----

for j in range(size):
    coluna = {
        solucao[i][j]
        for i in range(size)
    }

    if coluna != valores_esperados:
        return False

# -----
# Blocos n x n
# -----

for bloco_i in range(n):
    for bloco_j in range(n):

        valores_bloco = []

        for di in range(n):
            for dj in range(n):
                i = bloco_i * n + di
                j = bloco_j * n + dj

                valores_bloco.append(
                    solucao[i][j]
                )

            if set(valores_bloco) != valores_esperados:
                return False

# -----
# Preservação das pistas
# -----

for (i, j), val in pistas.cells.items():
    if val is not None:
        if solucao[i][j] != val:
            return False

return True

```

```

In [ ]: # =====
# Execução controlada de R6 para n = 2.
# Grelha: 4 x 4
# =====

solucao_2, pistas_2 = resolver_sudoku(
    n=2,
    k=2,
    seed=123
)

print("Pistas n=2:", pistas_2.cells)

```

```

if solucao_2 is None:
    print("n=2: INFEASIBLE")

else:
    print("Solução n=2:")

    for linha_solucao_2 in solucao_2:
        print(linha_solucao_2)

    assert validar_sudoku(
        solucao_2,
        2,
        pistas_2
    )

    print(
        "n=2: validação independente concluída com sucesso"
    )

```

Pistas n=2: {(0, 1): 1, (1, 0): 4}

Solução n=2:

[3, 1, 2, 4]

[4, 2, 1, 3]

[2, 3, 4, 1]

[1, 4, 3, 2]

n=2: validação independente concluída com sucesso

```

In [ ]: # =====
# Execução controlada de R6 para n = 3.
# Grelha: 9 x 9
# =====

solucao_3, pistas_3 = resolver_sudoku(
    n=3,
    k=3,
    seed=123
)

print("Pistas n=3:", pistas_3.cells)

if solucao_3 is None:
    print("n=3: INFEASIBLE")

else:
    print("Solução n=3:")

    for linha_solucao_3 in solucao_3:
        print(linha_solucao_3)

    assert validar_sudoku(
        solucao_3,
        3,
        pistas_3
    )

    print(

```

```
) "n=3: validação independente concluída com sucesso"
```

Pistas n=3: {(0, 6): 7, (3, 7): 5, (1, 2): 2}

Solução n=3:

[5, 3, 8, 2, 9, 1, 7, 6, 4]

[1, 6, 2, 4, 7, 3, 5, 9, 8]

[7, 4, 9, 5, 6, 8, 2, 1, 3]

[2, 7, 3, 8, 4, 6, 1, 5, 9]

[4, 1, 5, 3, 2, 9, 8, 7, 6]

[9, 8, 6, 7, 1, 5, 4, 3, 2]

[6, 5, 4, 1, 3, 2, 9, 8, 7]

[3, 2, 1, 9, 8, 7, 6, 4, 5]

[8, 9, 7, 6, 5, 4, 3, 2, 1]

n=3: validação independente concluída com sucesso

```
In [ ]: # =====
# Contribuição LLM:
# Testes adicionais de cobertura para R4 e para o validador
# independente de R6. Estes testes reforçam a cobertura e não
# introduzem novos requisitos funcionais.
# =====

# -----
# 1. R4 - caso limite  $k = 0$ 
# -----

pistas_zero = gerar_pistas(
    n=2,
    k=0,
    seed=123
)

assert isinstance(pistas_zero, box)
assert len(pistas_zero.cells) == 0

# -----
# 2. R4 - caso limite  $k = n^4$ 
# Para  $n = 2$  existem  $4 \times 4 = 16$  células.
# -----

pistas_todas = gerar_pistas(
    n=2,
    k=16,
    seed=123
)

assert isinstance(pistas_todas, box)
assert len(pistas_todas.cells) == 16

coordenadas_esperadas = {
    (i, j)
    for i in range(4)
    for j in range(4)
```

```

}

# Todas as 16 células foram selecionadas exatamente uma vez.
assert set(pistas_todas.cells.keys()) == coordenadas_esperadas

# Todos os valores estão no domínio 1..4.
assert all(
    1 <= val <= 4
    for val in pistas_todas.cells.values()
)

# -----
# 3. Teste negativo de validar_sudoku
#   Reutiliza a solução válida obtida anteriormente para n = 2.
# -----

assert solucao_2 is not None
assert validar_sudoku(solucao_2, 2, pistas_2)

# Cópia independente da matriz.
solucao_2_invalida = [
    linha[:] for linha in solucao_2
]

# Introduz deliberadamente uma repetição na primeira linha.
solucao_2_invalida[0][0] = solucao_2_invalida[0][1]

assert validar_sudoku(
    solucao_2_invalida,
    2,
    pistas_2
) is False

print("Testes adicionais de cobertura concluídos com sucesso")

```

Testes adicionais de cobertura concluídos com sucesso

Eficiência e Escala

```

In [ ]: # =====
# Contribuição LLM:
# Experiência reprodutível de eficiência e escala.
# Mede apenas resolver_sudoku(), para n = 2, 3 e 4,
# com k = n e seeds 100, 101 e 102.
#
# As variáveis da experiência são locais à função para evitar
# conflitos de nomes entre células do Marimo.
# =====

def executar_experiencia_desempenho():
    import time

    dimensoes = [2, 3, 4]

```

```

seeds = [100, 101, 102]

resultados = []

# -----
# Execuções
# -----

for n_local in dimensoes:
    for seed_local in seeds:

        # Mede APENAS resolver_sudoku()
        inicio_local = time.perf_counter()

        solucao_local, pistas_local = resolver_sudoku(
            n=n_local,
            k=n_local,
            seed=seed_local
        )

        fim_local = time.perf_counter()

        tempo_local = fim_local - inicio_local

        # Validação feita fora do intervalo medido
        if solucao_local is None:
            estado_local = "INFEASIBLE"
            validada_local = None
        else:
            estado_local = "SOLUÇÃO"
            validada_local = validar_sudoku(
                solucao_local,
                n_local,
                pistas_local
            )

        resultados.append({
            "n": n_local,
            "dimensao": f"{n_local**2}x{n_local**2}",
            "seed": seed_local,
            "tempo_s": tempo_local,
            "estado": estado_local,
            "validada": validada_local
        })

# =====
# 1. Resultados individuais
# =====

print("RESULTADOS INDIVIDUAIS")
print("-" * 85)

for resultado in resultados:
    print(
        f"n={resultado['n']} | "
        f"grelha={resultado['dimensao']} | "

```

```

        f"seed={resultado['seed']} | "
        f"tempo={resultado['tempo_s']:.6f} s | "
        f"estado={resultado['estado']} | "
        f"validada={resultado['validada']}]"
    )

# =====
# 2. Resumo por dimensão
# =====

print()
print("RESUMO POR DIMENSÃO")
print("-" * 85)

for n_local in dimensoes:
    resultados_n = [
        resultado
        for resultado in resultados
        if resultado["n"] == n_local
    ]

    tempos_n = [
        resultado["tempo_s"]
        for resultado in resultados_n
    ]

    numero_execucoes = len(resultados_n)

    numero_solucoes = sum(
        resultado["estado"] == "SOLUÇÃO"
        for resultado in resultados_n
    )

    numero_inviaveis = sum(
        resultado["estado"] == "INFEASIBLE"
        for resultado in resultados_n
    )

    tempo_medio = sum(tempos_n) / numero_execucoes
    tempo_minimo = min(tempos_n)
    tempo_maximo = max(tempos_n)

    print(
        f"n={n_local} | "
        f"grelha={n_local**2}x{n_local**2} | "
        f"execuções={numero_execucoes} | "
        f"tempo médio={tempo_medio:.6f} s | "
        f"mínimo={tempo_minimo:.6f} s | "
        f"máximo={tempo_maximo:.6f} s | "
        f"SOLUÇÃO={numero_solucoes} | "
        f"INFEASIBLE={numero_inviaveis}"
    )

return resultados

```

```
executar_experiencia_desempenho()
```

RESULTADOS INDIVIDUAIS

```
-
n=2 | grelha=4x4 | seed=100 | tempo=0.024418 s | estado=SOLUÇÃO | validade=True
n=2 | grelha=4x4 | seed=101 | tempo=0.021688 s | estado=SOLUÇÃO | validade=True
n=2 | grelha=4x4 | seed=102 | tempo=0.018364 s | estado=SOLUÇÃO | validade=True
n=3 | grelha=9x9 | seed=100 | tempo=0.003253 s | estado=INFEASIBLE | validade=None
n=3 | grelha=9x9 | seed=101 | tempo=0.115156 s | estado=SOLUÇÃO | validade=True
n=3 | grelha=9x9 | seed=102 | tempo=0.003543 s | estado=INFEASIBLE | validade=None
n=4 | grelha=16x16 | seed=100 | tempo=0.673091 s | estado=SOLUÇÃO | validade=True
n=4 | grelha=16x16 | seed=101 | tempo=0.464242 s | estado=SOLUÇÃO | validade=True
n=4 | grelha=16x16 | seed=102 | tempo=0.660489 s | estado=SOLUÇÃO | validade=True
```

RESUMO POR DIMENSÃO

```
-
n=2 | grelha=4x4 | execuções=3 | tempo médio=0.021490 s | mínimo=0.018364 s | máximo
=0.024418 s | SOLUÇÃO=3 | INFEASIBLE=0
n=3 | grelha=9x9 | execuções=3 | tempo médio=0.040651 s | mínimo=0.003253 s | máximo
=0.115156 s | SOLUÇÃO=1 | INFEASIBLE=2
n=4 | grelha=16x16 | execuções=3 | tempo médio=0.599274 s | mínimo=0.464242 s | máxi
mo=0.673091 s | SOLUÇÃO=3 | INFEASIBLE=0
```

Resultados e análise

Foi realizada uma experiência simples de eficiência e escala para $n = 2$, $n = 3$ e $n = 4$. Em cada dimensão foram efetuadas três execuções, usando $k = n$ e as sementes determinísticas 100, 101 e 102.

Estas escolhas correspondem a uma decisão metodológica do grupo para ilustrar o comportamento da implementação e não a um requisito específico do enunciado.

O tempo foi medido com `time.perf_counter()` apenas em torno da chamada a `resolver_sudoku()`. A validação da solução através de `validar_sudoku()` foi realizada depois da medição e, por isso, não está incluída nos tempos apresentados.

Resultados observados

n	Grelha	Seed	Tempo (s)	Estado	Validação
2	4×4	100	0.024418	SOLUÇÃO	True
2	4×4	101	0.021688	SOLUÇÃO	True
2	4×4	102	0.018364	SOLUÇÃO	True
3	9×9	100	0.003253	INFEASIBLE	—
3	9×9	101	0.115156	SOLUÇÃO	True
3	9×9	102	0.003543	INFEASIBLE	—

n	Grelha	Seed	Tempo (s)	Estado	Validação
4	16×16	100	0.673091	SOLUÇÃO	True
4	16×16	101	0.464242	SOLUÇÃO	True
4	16×16	102	0.660489	SOLUÇÃO	True

Resumo por dimensão

n	Grelha	Execuções	Média (s)	Mínimo (s)	Máximo (s)	SOLUÇÃO	INFEASIBLE
2	4×4	3	0.021490	0.018364	0.024418	3	0
3	9×9	3	0.040651	0.003253	0.115156	1	2
4	16×16	3	0.599274	0.464242	0.673091	3	0

Observações dos dados

Para $n = 2$, as três instâncias tiveram solução e foram validadas independentemente. Os tempos observados variaram entre 0.018364 s e 0.024418 s.

Para $n = 3$, apenas a instância com `seed = 101` teve solução, demorando 0.115156 s. As instâncias com `seed = 100` e `seed = 102` foram declaradas **INFEASIBLE** e terminaram em 0.003253 s e 0.003543 s, respectivamente.

Para $n = 4$, as três instâncias tiveram solução e foram validadas, com tempos entre 0.464242 s e 0.673091 s.

Interpretação

A média obtida para $n = 3$ não deve ser comparada diretamente com as médias de $n = 2$ e $n = 4$, uma vez que duas das três instâncias foram **INFEASIBLE** e terminaram muito rapidamente.

Nas instâncias satisfazíveis observadas, as execuções para $n = 4$ foram mais demoradas do que as execuções satisfazíveis realizadas para $n = 2$ e $n = 3$.

A variabilidade observada entre as execuções sugere que o tempo de resolução pode depender não apenas da dimensão da grelha, mas também das restrições concretas introduzidas pelas pistas.

No entanto, foram utilizadas apenas três sementes por dimensão. Estes resultados caracterizam apenas as instâncias efetivamente executadas e não permitem estabelecer conclusões gerais sobre a complexidade do problema nem extrapolar o desempenho para dimensões não experimentadas.

Limitações

A geração de pistas atribui posições e valores aleatoriamente sem garantir antecipadamente que o conjunto resultante seja compatível com algum Sudoku. Por decisão de implementação do grupo, quando as pistas tornam o CSP inviável, o solver reporta `INFEASIBLE` e não são geradas automaticamente novas pistas.

O modelo procura uma solução que satisfaça todas as restrições, mas não verifica a unicidade dessa solução. Assim, uma instância pode admitir várias soluções válidas, sendo devolvida uma solução encontrada pelo solver.

A experiência de desempenho foi limitada a `n = 2`, `n = 3` e `n = 4`, com `k = n` e três sementes por dimensão (`100`, `101` e `102`).

Consequentemente, os tempos apresentados dizem respeito apenas às instâncias executadas e não devem ser extrapolados para dimensões ou configurações que não foram experimentadas.

Reprodutibilidade

O ambiente utilizado no desenvolvimento e nas execuções finais foi:

- Python 3.14.7
- Marimo 0.25.0
- OR-Tools 9.15.6755

O cabeçalho do notebook mantém o requisito original:

```
requires-python = ">=3.14"
```

e declara explicitamente as dependências utilizadas:

- `marimo==0.25.0`
- `ortools==9.15.6755`

Durante o desenvolvimento foi necessário corrigir a configuração das dependências. O problema ficou resolvido depois de declarar explicitamente `ortools` nas dependências do notebook, mantendo o requisito original `requires-python >= 3.14`.

A aleatoriedade da geração de pistas pode ser controlada através do parâmetro `seed`.

As execuções controladas de R6 utilizaram:

- `n = 2`, `k = 2`, `seed = 123`;
- `n = 3`, `k = 3`, `seed = 123`.

A experiência de eficiência e escala utilizou:

- `n = 2` , `n = 3` e `n = 4` ;
- `k = n` ;
- sementes `100` , `101` e `102` ;
- três execuções por dimensão.

As soluções obtidas são verificadas através de `validar_sudoku()` , um validador independente do solver que verifica a dimensão da matriz, todas as linhas, todas as colunas, todos os blocos e a preservação das pistas fixadas.

```
In [ ]: # =====
# Registo do ambiente utilizado na execução final.
# =====

def registrar_ambiente():
    import sys
    import marimo
    import ortools

    print("AMBIENTE DE EXECUÇÃO")
    print("-" * 50)
    print(f"Python: {sys.version}")
    print(f"Marimo: {marimo.__version__}")
    print(f"OR-Tools: {ortools.__version__}")

registrar_ambiente()
```

AMBIENTE DE EXECUÇÃO

Python: 3.14.7 | packaged by Anaconda, Inc. | (main, Aug 14 2026, 10:12:46) [MSC v.1942 64 bit (AMD64)]
Marimo: 0.25.0
OR-Tools: 9.15.6755

Matriz final de rastreabilidade

A tabela seguinte relaciona cada requisito com a implementação correspondente e com a evidência de teste efetivamente executada.

Requisito	Implementação	Evidência real de teste	Estado
R1 — grupo genérico	<code>box</code> , <code>add()</code> , <code>to_matrix()</code>	Criação vazia; construtor com células iniciais; célula livre; célula fixa; representação matricial; coordenadas inválidas; valores inválidos; coordenada duplicada	Implementado e testado
R2 — blocos	<code>cube</code>	Construção de blocos diferentes e rejeição de índices de bloco inválidos	Implementado e testado

Requisito	Implementação	Evidência real de teste	Estado
R3 — percursos	<code>path</code>	Horizontal crescente e decrescente; vertical crescente e decrescente; <code>inicio == fim</code> ; percurso não horizontal/vertical; coordenadas fora da grelha	Implementado e testado
R4 — pistas aleatórias	<code>gerar_pistas()</code>	Resultado do tipo <code>box</code> ; número de pistas; coordenadas e valores válidos; reprodução com a mesma seed; valor por omissão; valores inválidos de <code>k</code> ; <code>k = 0</code> ; <code>k = n^4</code> para <code>n = 2</code> , cobrindo exatamente as 16 células	Implementado e testado
R5 — modelo CSP	<code>sudoku_csp</code> , <code>add_groups()</code> , <code>solve()</code>	Caso satisfazível; solução devolvida como matriz; <code>AllDifferent</code> ; preservação de valores fixos; grupos de diferentes tipos tratados uniformemente; grupo com <code>n</code> incompatível rejeitado; CSP contraditório devolvendo <code>None</code> após <code>INFEASIBLE</code>	Implementado e testado
R6 — Sudoku completo	<code>resolver_sudoku()</code> e <code>validar_sudoku()</code>	Fluxo completo executado para <code>n = 2</code> e <code>n = 3</code> ; validação independente de linhas, colunas, blocos e pistas; teste negativo em que uma solução foi deliberadamente corrompida e rejeitada pelo validador	Implementado e validado
Eficiência e escala	Experiência com <code>time.perf_counter()</code>	Nove execuções: <code>n = 2, 3, 4</code> , três sementes por dimensão, com registo de tempo, estado e validação	Executado e documentado

Utilização e contribuição da LLM

Foram utilizadas ferramentas LLM como apoio à interpretação dos requisitos, formalização, implementação, criação de testes, análise dos resultados e organização da documentação.

As componentes substanciais propostas com apoio da LLM encontram-se identificadas através de comentários no código e são resumidas na tabela seguinte.

Componente	Contribuição da LLM	Contribuição do grupo
Interpretação e formalização de R1–R6	Organização dos requisitos, definição das variáveis, restrições e correspondência entre requisitos e implementação	Revisão das propostas, discussão das decisões e aprovação da modelação

Componente	Contribuição da LLM	Contribuição do grupo
<code>box</code>	Proposta inicial da implementação e dos testes de R1	Definição das decisões de implementação, integração no Marimo e execução dos testes
<code>cube</code> e <code>path</code>	Proposta inicial das implementações e baterias de testes de R2 e R3	Revisão, integração e execução dos testes
Geração de pistas	Proposta de <code>gerar_pistas()</code> e testes de R4	Decisões sobre reprodução por seed e comportamento perante inviabilidade; integração e execução
Modelo CP-SAT	Formalização e proposta da implementação de <code>sudoku_csp</code> , <code>add_groups()</code> e <code>solve()</code>	Escolha e confirmação do OR-Tools CP-SAT, integração, configuração do ambiente e execução dos testes
Construção do Sudoku	Proposta de <code>resolver_sudoku()</code>	Integração no notebook e execução do fluxo completo
Validação independente	Proposta de <code>validar_sudoku()</code>	Execução da validação sobre as soluções reais e teste negativo com solução deliberadamente corrompida
Testes adicionais	Proposta dos testes limite $k = 0$, $k = n^4$ e teste negativo do validador	Execução e confirmação dos resultados reais
Eficiência e escala	Proposta da metodologia e do código de medição com <code>time.perf_counter()</code>	Escolha dos parâmetros da experiência, execução das nove instâncias e fornecimento dos tempos reais
Análise e documentação	Apoio à organização da análise de desempenho, limitações, reprodutibilidade, rastreabilidade e conclusão	Revisão final do conteúdo e responsabilidade pelos resultados efetivamente apresentados

O grupo integrou o código no notebook Marimo, tomou as decisões de implementação, resolveu os problemas de configuração do ambiente e de organização das células, executou todos os testes e experiências e forneceu os resultados reais utilizados neste relatório.

Diálogos LLM

Diálogo principal utilizado no desenvolvimento do TP1.2 — Sudoku:

[Diálogo principal TP1.2 — Sudoku](#)

Conclusão

Neste exercício foi modelado um Sudoku genérico $n^2 \times n^2$ como um problema de satisfação de restrições.

A classe `box` fornece a abstração genérica para grupos de células, enquanto `path` e `cube` permitem construir linhas, colunas e blocos. O modelo implementado com OR-Tools CP-SAT associa uma variável inteira a cada célula e aplica genericamente restrições `AllDifferent` e igualdades correspondentes aos valores fixos.

Os requisitos R1–R6 foram implementados e testados. O fluxo completo foi executado e validado para `n = 2` e `n = 3`, e a experiência de eficiência e escala incluiu também `n = 4`.

A implementação é parametrizada em `n` e não contém qualquer caso especial para uma grelha 9×9.